

Against service layers in Django

 [March 16, 2020](#)  [Django](#), [Python](#)

This post now has [a followup](#).

Recently I've seen posts and questions pop up in a few places about [a sort of “enterprise” Django style guide](#) that's been getting attention. There are a number of things I disagree with in that guide, but the big one, and the one people have mostly been asking about, is the recommendation to add a “service layer” to Django applications. The short version of my opinion on this is: it's probably not what you want in Django apps.

The longer version follows below.

What is a service layer, anyway?

First it's helpful to understand what's meant by “service” and what the theoretical goal is, so let's take an example. Suppose you have a Django-powered blog app, with an `Entry` model representing entries in the blog. The standard way to query for entries would be something like:

```
from blog.models import Entry
entries = Entry.objects.all()
```

This would get you a `QuerySet` of all entries.

The “service” approach would be to do something like:

```
from blog import service
entries = service.get_list()
```

Where the `service` module implements a variety of functions/methods that, under the hood, call various Django `ORM` methods to actually perform queries and manipulate model objects. Typical “service” implementations provide at least things like `get()`, `list()`, `create()`, `update()`, and `delete()` (or other synonyms for those operations), and often also provide more complex “business logic” methods. In a simple case, things like “publishing” a blog entry by changing its status from draft to live. More complex data models and logic will end up with more complex service layers.

Quick aside: the earliest Django ORM was actually a code generator that would leave behind one auto-generated module per model class, very similar to the type of `service` module mentioned here, containing all the query methods for working with that model. For the first public release it was still there, but switched to generating the modules in-memory rather than leaving them on the filesystem. This was confusing and nearly universally frowned upon; if you want an overview of why, skim to the second section of [this old post](#), and if you want to know how something like that even gets implemented, I walk through a simplified example [in this other old post](#). Anyway, Django 0.95 (released in July 2006) replaced the code generator with the initial implementation of the modern Django ORM.

Meanwhile, the motivations for the service-layer approach mostly come down to two things:

1. Provide a clear place to put “business logic” — common queries and operations that will need to be performed repeatedly on or with instances of the data models.
2. Hide the underlying implementation from all other code, so that you could swap it out and replace with something else and not have to rewrite any code other than the “service”.

General problems with “service” approaches

I have several general objections to this approach that apply beyond just the use case of Django.

One is that this is taking a pretty common principle of “enterprise” architecture and applying it without careful consideration. The general idea is to try to put layers of indirection/abstraction in front of various components, so that the actual implementations of those components can be changed without breaking other code.

There certainly are times and places when this is a useful thing to do, but there are also times and places when it isn’t useful. A typical web application does involve a lot of components, and it is the case that any application that gets large enough or lasts long enough will eventually want to change/swap some of them. But effort should be focused on the ones that are likely to *actually* get changed/swapped, rather than on some sort of reflex action of always putting layers of indirection in front of everything without pausing to think about whether it’s really useful (see also: [YAGNI](#)).

Since I'm going to talk specifically about Django in the next section I'll use it as an example here: a lot of parts of Django are swappable out of the box in ways that are meant to minimize code changes. Caching layers, session storage, authentication sources, file-storage implementations, logging systems, email-sending, templating languages... the list of things you can easily swap, because they've been put behind configuration and generic/indirect APIs, is pretty long. But it's also a list that's informed by experience and actual practice. Or, in other words, it's a list of things that commonly *do* get changed/swapped over the life of a lot of real projects. Other components, like the forms library, the admin interface, and so on are effectively optional, and often will be dropped for something else, or just not even used in the first place.

But what about the data modeling and access component (which in Django's case is its ORM)? In theory, hiding it behind a service layer means you can change it out for something else without rewriting other code. But how often does that really happen in practice? My experience — and I've been working in this industry for over 15 years at places both big and small, and been through some pretty major platform migrations — is that it happens almost never. Swapping something as major as how you model and persist/retrieve your core data is not something you do lightly, and typically only happens at a time when you're making other massive changes — completely switching platform/language/framework, or otherwise doing a ground-up rewrite of the entire system. And, ironically, it's in the kinds of organizations *most* likely to insist on “enterprise” architecture patterns that the payoff is *least* likely ever to be realized, because such organizations are almost fanatically averse to the kind of change this architecture is meant to enable. See, for example, all the places for which we know a decade (or longer) is not even close to enough time to do platform upgrades.

A second problem is that when you decide to go the “service” route, you are changing the nature of your business. This is related to an argument I bring up occasionally when people tell me they don't use “frameworks” and never will: what they actually mean, whether they realize it or not, is “we built and now have to maintain and train our developers on our own ad-hoc private framework, on top of whatever our normal business is”. And adopting the service approach essentially means that, whatever your business was previously, now your business is that plus developing and maintaining something close to your own private ORM.

This is a necessary consequence, because sooner or later you're going to need a significant fraction of the features of whatever your *real* data layer is, which means you will have to implement sufficient features in your service layer to be able to take advantage of them. And I'd bet a lot of money that most places which do this do *not* take the necessary level of care to ensure their reimplementations of those features is truly

generic enough to still allow swapping out the underlying implementation. Designing good and thoughtful APIs for interacting with your data is difficult and time-consuming. What's much easier, and can be done quickly to clear out your cards for the current sprint, is copying someone else's already-designed API, which means the service layer, over time, almost always ends up with an API that's tightly coupled to whatever the underlying data layer's API is, and there goes a lot of the claimed benefit of the service layer out the window.

Yet another issue is that this is a solved problem, and the solution is not service layers. If you want a clear separation of “business logic” and “data access”, and/or believe you need the benefits that you think will come from decoupling them, the solution is [the Data Mapper ORM pattern](#). Writing a “service” to hide your real data-access layer is basically just Data Mapper with extra steps and without being able to take advantage of off-the-shelf implementations. And while you will have a theoretically harder time swapping it out for some other underlying component, I've already mentioned the problem with trying to optimize for that.

Now, about Django...

Some of the general objections above apply even more strongly to the specific case of Django. For example: in general, people don't often swap out their data access layer without also doing other massive rewrites. And in Django-based applications it's even less likely to try to swap out the ORM without other huge code changes happening at the same time, because the Django ORM is probably the single most tightly-integrated component of the entire framework — if you stop using it, you're throwing away so much other stuff that “why are we even still using Django” becomes a really significant question.

And I said above that I think the real solution, if you feel like you need the decoupling of “business logic” and data access/modeling, is a Data Mapper ORM. Well, Django's ORM is not that; it's very much an [Active Record](#) ORM, and trying to use it in a Data Mapper style is going to result in a lot of unhappiness. You never really want to be fighting against your framework that way.

Worse still is the issue of “service” layers requiring you to basically build your own ORM. To really do a backend-agnostic service layer on top of the Django ORM, you need to replace or abstract away some of its most fundamental and convenient abstractions. For example, most of the commonly-used ORM query methods return either instances of your model classes, or instances of Django's `QuerySet` class (which is a kind of chained-

API results wrapper around a query). In order to avoid tightly coupling to the structure and API of those Django-specific objects, your service layer needs to translate them into something else — likely generic iterables to replace `QuerySet`, and some type of “business object” instance to replace model-class instances. Which is a non-trivial amount of work even in patterns like Data Mapper that are designed for this, and even more difficult to do in an Active Record ORM that isn’t.

And in the process of abstracting away those underlying ORM objects and interfaces, you also are throwing away a lot of their utility. `QuerySet` is already a really cool (no bias on my part here, of course) caching query wrapper with a bunch of functionality to turn common use patterns into efficient query behavior. When you abstract away its existence, you either lose that or have to reimplement it yourself.

You’ll also lose, or have to abstract/reimplement, a lot of the integrations between the ORM and other parts of Django — no more handy generic views, no more helpful auto-generating forms or (if you’re using DRF) serializers, and so on, because those all break the “service” abstraction by exposing other code to the existence and APIs of the underlying ORM.

What to do instead

I’ve said a few times in passing, and covered in more detail in the “Mastering the Django ORM” tutorial I’ve presented a few times at conferences, that (in my opinion) in most well-designed Django applications, the models — and potentially associated utility code, like custom `Manager` or `QuerySet` subclasses — *are* the API exposed to other code. Which in turn means that they are the place where the “business logic” should be implemented.

The ORM itself already provides the common query patterns you’d implement in a “service” layer, so you don’t need to reimplement them. When going beyond those, here’s a quick set of recommendations for how to implement things:

- If you need an alternate constructor for model instances, do it as a manager method (not as a `classmethod` on the model, or in some other class or code elsewhere). In my ORM tutorial I use the example of a model that stores an RGB color as three integer fields, and implementing a manager method that can build an instance from a hexadecimal value, like `RGBColor.objects.from_hex('#000080')`.
- If you have a custom, complex and/or often-used query against the model’s entire table, do it as either a manager method (if you think you’ll never need to use it

chained after other query methods), or as a `QuerySet` method (if you will need to be able to chain it after other things). And remember you can write a `QuerySet` subclass with your custom method(s) and then [auto-generate a model manager from it](#).

- If you need more complex fetching of related objects from a single model instance, do it as a method on that model class.
- Any sort of logical operation on only a single model instance should be a method on the model class.
- Any sort of logical operation on a set of instances of the same model should be a method on the model's manager or its `QuerySet`.

If this sounds complex, it's just because I've written out a bunch of specific examples — the base pattern is “things that involve one model instance go on the model class; things involving multiple or all instances go on the model's `QuerySet` or manager”.

And in general, other code should obey the [Law of Demeter](#) with respect to an application's models. The traditional “one-dot rule” version is a bit inflexible for a language like Python (where lookups often start with `self`), and for the specific model/manager/query method architecture used in the Django ORM, so if you want an exhaustive version of it, code external to a Django model class should restrict itself to accessing:

- On an instance of the model, any custom public methods, and `get_absolute_url()`. Access to fields (including relationship traversals) for reading should be discouraged, and for writing should be forbidden. Access to `save()` and `delete()` should similarly be off-limits.
- From an instance of a related model, the same as above (essentially allowing a “two-dot” access for some cases).
- On the model class itself, the default constructor, the manager, its top-level object-creation methods (`create()`, `get_or_create()`) and any alternate constructors or custom top-level query methods, but generally *not* built-in query methods other than `all()` (again, allowing limited “two-dot” accesses). The one exception here is for using generic views that need access to a broader range of the query methods (like `filter()`) to do their work; views that are specific to a single model class should be making use of purpose-built query methods on that model's manager or `QuerySet` instead of embedding their own complex query logic, or retrieving a single instance and then invoking public methods on it that implement the desired logical operations.

This usually means views should contain as little “business logic” as possible, instead deferring to the model for that. In the tutorial I give the example of a bug-tracking system where a view that resolves a ticket is implemented by querying for the `Ticket` instance, then manually setting a bunch of its fields and calling `save()` — then contrast it with an implementation where `resolve()` is implemented as a method on `Ticket`, so all the view has to do is query up the instance and then call its `resolve()` method. The second version is almost always going to be the right way to do things (and not just in Django). As the old joke goes, when you want to walk the dog, you shouldn’t reach down and grab and pick up and put down each of its legs manually in order; you should just trust that the dog knows how to `walk()` on its own. This is especially true with Active Record ORMs like Django’s.

Speaking of which: following the Law of Demeter is a lot harder, and often just plain impossible, in a “service” approach, because the service layer takes responsibility for implementing “business logic” operations like resolving the `Ticket` or walking the `Dog`, and so has to have detailed knowledge of the structure of those objects and how to reach into them and manipulate their fields (if it didn’t, and those objects instead exposed their own logical `resolve()`, `walk()`, etc. operations as methods, there’d be no point to the service layer, because the “business logic” would already be implemented elsewhere!).

Go forth, and write service layers no more

I’ve mentioned a couple times above that I’m presenting my opinions, and usually at the end of a post like this I make a point of reiterating that and saying that if something else works for you, do the thing that works for you. And it’s still true that what I’ve said above is just my opinion, and you shouldn’t necessarily take it as a set of ironclad rules you have to follow.

But in this case I’m going to modify my usual statement slightly: the above are not just my random personal preferences. They are opinions, but they’re informed opinions from a *lot* of time spent working on and around large codebases (many in Django, some not), and from seeing what worked and what didn’t. My experience is that attempts to build service layers usually fall into the “didn’t work” category, and that sticking mostly to the guidelines I’ve laid out above for how to work with models and “business logic” in Django is more likely to wind up in the “worked” category.

And if, after reading this, you still are convinced that a service-layer approach with business logic separate from the data models is the right thing for you, then go for it, but

I'll urge you one last time to at least strongly consider *not* doing it with Django, and instead building on top of a component stack that uses a Data Mapper ORM (in the Python world, [SQLAlchemy](#) is far and away the best choice) that natively incorporates that type of separation. In the long term, I think that's going to be a much happier and more productive path than trying to fit the square peg of a service layer into the round hole of Django's Active Record approach.